Dev Process Primer

What the evidence says about building software well, with and without AI, and how fomodoro applies it. Prepared November 2026.

How to read this. Each idea gets a plain-English definition, an example from our project (in green), a badge saying how solid the evidence is, and numbered citations [n] pointing to the Sources at the end. The aim is not to memorize practices but to know *why* each one is in our process and how much weight it can bear. Some of our choices rest on decades of research; others are sensible practitioner habits that nobody has tested experimentally. It helps to know which is which.

STRONG	Replicated empirical research or a formal standard.
MODERATE	Some empirical support, but limited, dated, or contested.
HEURISTIC	Expert practitioner guidance, not experimentally established.

1. Why process at all

Process is the set of habits that turns "I wrote some code" into "working software is in users' hands, and I can change it safely tomorrow." For a solo project it is tempting to skip it. The research below is the case for not skipping it, and for keeping it light.

The DORA research and the four key metrics STRONG

DORA (DevOps Research and Assessment) is a long-running research program, now run by Google Cloud, that surveys tens of thousands of technology professionals each year. The book *Accelerate* summarized its first four years [1]. Its central claim: you can measure software delivery performance with a handful of numbers, and teams that score well on them also tend to do better as organizations. The original four metrics were:

- **Deployment frequency:** how often you ship to production.
- **Lead time for changes:** how long from a commit to that code running in production.
- **Change failure rate:** what share of deployments cause a problem that needs fixing.
- **Time to restore:** how long it takes to recover when something breaks.

The surprising finding was that speed and stability are not a trade-off: the best performers are both faster *and* more stable. DORA has since refined the set into two groups, **throughput** (lead time, deployment frequency, failed-deployment recovery time) and **instability** (change fail rate, and a newer deployment rework rate) [2].

For fomodoro: a merged PR deploys to Cloudflare Pages automatically, so lead time is "how long a PR sits open." Change failure rate is "how often a merge forces a follow-up fix." We don't need a dashboard; the monthly log can note both in a line.

Caveat: DORA's data are surveys, so they show strong, repeated associations rather than controlled cause and effect. The pattern has held across many years and many thousands of respondents, which is why it earns a Strong badge.

Small batches STRONG

Working in small batches means slicing work so each change is small enough to finish, review, and ship quickly. DORA lists it as a core capability: smaller changes give faster feedback, are easier to debug when they fail, and make it easier to abandon an idea that isn't working instead of sinking more effort into it [3]. Most of the rest of this primer is small batches applied to different things: branches, reviews, tests, AI sessions.

One GitHub Issue per PR, and one issue per Claude session. "Add the settings screen" becomes three issues: durations, sounds, and the 2FA toggle.

AI as an amplifier (DORA 2025) MODERATE

DORA's 2025 report was retitled *State of AI-assisted Software Development* and drew on nearly 5,000 survey responses plus over 100 hours of interviews [4]. Its headline finding, in the authors' own words: "AI doesn't fix a team; it amplifies what's already there" [5]. Teams with good foundations (automated tests, mature version control, fast feedback, small batches) got more out of AI; teams without them saw AI make existing problems worse. The report found AI adoption now goes along with higher throughput, but still with *more* delivery instability: AI produces more change, and weak downstream checks let that extra change break things [5]. The accompanying AI Capabilities Model names seven practices that magnify AI's benefits, including working in small batches, strong version control practices, and quality internal platforms [3][4].

This is the strongest argument for our CI gates. Claude can write a lot of code quickly. The gates are what keep "a lot of code quickly" from becoming "a lot of bugs quickly."

Why Moderate: one year of survey data, a new framing, and correlational design. It fits the earlier DORA findings well, but it has not had time to be replicated.

The METR randomized trial MODERATE

In 2025 the research group METR ran a randomized controlled trial: 16 experienced open-source developers worked on 246 real issues in large, mature repositories they knew well, and each issue was randomly assigned to "AI allowed" or "AI not allowed" [6]. With AI allowed (mostly Cursor with Claude 3.5/3.7 Sonnet), they took about **19% longer**. Before starting they had predicted a 24% speedup, and even afterward they believed AI had sped them up by about 20% [7].

What it shows: self-reported productivity can be badly wrong, in the optimistic direction. Feeling fast is not evidence of being fast. **What it does not show:** the authors explicitly say it does not prove AI slows down most developers, that it doesn't represent most software work, and that AI may still help people in unfamiliar codebases or with less experience [7]. It used early-2025 tools; METR's page now marks the results as out of date and points to newer follow-up work [7].

The lesson for us is measurement, not abstinence. If we want to know whether a Claude workflow helps, the honest signal is the log (time from issue opened to PR merged, and how often a merge needed a fix), not how productive the session felt.

Why Moderate: a genuinely randomized design, which is rare and valuable, but small (16 people), a narrow setting, and tools that have since changed.

2. The walking skeleton

What it is HEURISTIC

A **walking skeleton** is the smallest possible version of the whole system that works end to end. It does almost nothing useful, but every major part exists and is connected: the user interface talks to the logic, the logic talks to the backend, the code is built, tested, and deployed by the real pipeline to the real hosting. Alistair Cockburn named the idea in the 1990s and made it one of the core strategies in his book *Crystal Clear* [8]. The metaphor: a skeleton has all its bones in the right places and can walk, even though it has no muscle yet. Later work adds flesh to bones that already exist, rather than building organs separately and hoping they fit together.

Steve Freeman and Nat Pryce made the idea central to test-driven development in *Growing Object-Oriented Software, Guided by Tests* [9]. Their version: before writing the first real feature, build the thinnest slice of functionality you can *automatically build, deploy, and test end to end*. That slice is where your first end-to-end test lives. Only then do you start on features, each one driven by a failing end-to-end test that runs through the skeleton.

Why it works

- **It flushes out integration risk first.** The hardest problems in a new project are often not the features but the joins: auth config, environment variables, CORS, deploy settings, a service worker that caches the wrong thing. A skeleton hits all of them in week one, when they are cheap to fix.
- **It makes "done" mean "deployed."** From day one, every change goes through the full pipeline. There is never a scary "first deploy" at the end.
- **It gives every later feature a home.** Each new feature is a small change to a running system, which is exactly the small-batch pattern from section 1.
- **It forces architectural decisions early but cheaply.** You have to decide how the pieces talk to each other, but with so little code that changing your mind costs little.

Fomodoro's M0 as a walking skeleton. M0 is deliberately boring. Every bone, almost no flesh:

Bone	In M0	Added later
UI	One React page saying "fomodoro — Timer coming soon", installable as a PWA (manifest, service worker, icons).	The timer engine, ring, cycles, breaks (M1).
Backend	None yet.	Supabase auth, settings, push, handoff (M2–M4).
Hosting	Deployed to Cloudflare Pages, with a preview URL for each PR.	Nothing new; every feature ships through it.
Pipeline	Every CI gate running, even if each checks very little: typecheck, lint, format, unit tests, a Playwright smoke test against the built app, test-integrity, PR checks, npm audit, gitleaks.	Stryker on the engine (M1); more tests. The gates themselves barely change.
Process	Claude guardrails (permissions, hooks), AGENTS.md, dev log, STATUS.md, ADRs, merge protection, first issues and PRs.	The same loop, repeated.

If M0 is done, any later issue can go from spec to production without touching the pipeline. That is the whole point.

An honest gap. A strict walking skeleton connects *every* major architectural component, and ours leaves out the backend: Supabase first appears in M2. A purer M0 would include one real round trip to Supabase. We traded that for a smaller first milestone; the cost is that the first backend integration problems surface later. (This is a judgment call, not evidence. It's worth revisiting at the M0 retro.)

Tracer bullets HEURISTIC

Andy Hunt and Dave Thomas describe a close cousin in *The Pragmatic Programmer*: **tracer bullets** [10]. Tracer rounds glow so a gunner can see where shots land and adjust in real time. In code, a tracer bullet is a thin end-to-end path through the system that you keep and grow, as opposed to a **prototype**, which you build to answer one question and then throw away. A walking skeleton is essentially the first tracer bullet for a whole project.

A throwaway sketch to see whether a ring or a bar feels better for the countdown is a prototype. The M0 page and its pipeline, which stay and grow into the real timer, are a tracer bullet.

3. Small batches in practice: branches and review

Short-lived branches and trunk-based development MODERATE

Trunk-based development means everyone integrates into one main branch ("trunk") frequently, instead of keeping long-running feature branches. DORA's guidance: keep three or fewer active

branches, merge into trunk at least daily, and avoid code freezes and separate "integration phases" [11]. Martin Fowler's continuous integration article makes the same point from the other direction: a branch that doesn't reach mainline at least daily is only partly integrated, and the longer it lives, the more painful the eventual merge [12].

Our branches are short-lived: one issue, one branch, merged (or deleted) within a day or two. With one developer there is rarely more than one or two open at once, so we stay inside DORA's guidance while still using PRs.

Why Moderate rather than Strong: trunk-based development is a DORA capability with research behind it, but Fowler notes that pull-request workflows slow integration somewhat, and how the findings transfer to a solo developer using PRs as a gate is untested.

How big should a change be for review? MODERATE

The most-cited number comes from a 2006 case study at Cisco, written up by Jason Cohen, founder of the review-tool company SmartBear: about 2,500 reviews covering 3.2 million lines of code [13]. The reported pattern was that reviewers find defects well when looking at roughly **200 to 400 lines** at a time, and that their effectiveness drops when reviews are larger or go faster than about 500 lines per hour [14].

Why Moderate: one company, one product, one vendor with a commercial interest, almost twenty years old. The direction (smaller reviews are better reviews) matches everything else we know; the exact numbers deserve less trust.

Our soft limit: if a PR diff is past about 400 lines of real code (not lockfiles or generated files), split it. Claude can easily produce a 1,500-line PR in one go, which is precisely why the limit matters.

What code review actually finds MODERATE

Alberto Bacchelli and Christian Bird studied code review at Microsoft by observing developers, interviewing them, surveying them, and hand-classifying hundreds of review comments [15]. Developers said the main reason for review was finding bugs, but the comments showed that reviews found fewer defects than expected, and that most comments were about code improvement, understanding, and sharing knowledge. They also found that the hardest part of reviewing is simply *understanding the change*.

This reframes our review step. CI catches most mechanical bugs. The human review of a Claude PR is mainly about understanding: do I know what this change does, does it match the spec, and could I maintain it? If I can't explain a PR, I don't merge it, even if every gate is green.

4. Specs and plans

Spec-driven development with AI HEURISTIC

Spec-driven development means writing down *what* you want, and how you'll know it works, before asking an AI agent to write code. Two tools launched in 2025 made it popular. GitHub's open-source **Spec Kit** splits work into four phases (specify, plan, break into tasks, implement), with a human check between each; GitHub itself calls it an experiment [16]. AWS's **Kiro** editor produces three markdown files per feature (requirements, design, tasks) and has the agent code against the approved files rather than the chat history [17].

The underlying idea is old (write requirements first), but the reason it matters more with AI is new: an agent will confidently build *something* from a vague prompt, and a spec is the cheapest place to catch "that's not what I meant." It also gives a fresh session, or a reviewer, a written definition of done.

Our loop is spec → plan → TDD build → PR → merge → log entry. Specs live as markdown in the repo; the plan file is the handoff between a planning session and a building session.

Why Heuristic: as of this writing there is no controlled research showing spec-driven AI workflows outperform alternatives. It is promising, widely adopted, and still emerging.

Architecture Decision Records (ADRs) HEURISTIC

An ADR is a short, numbered document that records one significant decision: the context, the decision, and its consequences. Michael Nygard proposed the format in a 2011 blog post [18]. The value is mostly for the future: when someone (including future you, or an AI agent with no memory) wonders "why on earth did we do it this way?", the answer is written down, including the options you rejected.

"ADR 0003: Cloudflare Pages over Vercel for hosting" — two paragraphs on the context, the choice, and what we gave up. Claude can read it before suggesting a migration we already considered and rejected.

5. Testing

Test-driven development (TDD) MODERATE

TDD is the cycle of **red** (write a small failing test), **green** (write just enough code to pass it), **refactor** (clean up while the test keeps you safe). The evidence is more modest than its advocates sometimes claim. A 2013 meta-analysis by Rafique and Mišić pooled 27 studies and found a small improvement in external quality and little to no effect on productivity; the quality benefit was larger in industrial settings and on bigger tasks [19].

A 2017 study by Fucci and colleagues dug into *which part* of TDD matters [20]. Tracking 39 professionals' actual working patterns, they found that the outcomes were associated with working in **small, steady steps**, not with writing the test first. Their suggestion is that TDD's benefits may come mostly from the short-cycle rhythm it enforces.

We use TDD, but the part we're strict about is the short cycle: one behavior, one test, one small change. Test-first is our default because it keeps cycles short and forces the agent to show a failing test before it writes the fix, which also proves the test can fail.

Test pyramid vs testing trophy HEURISTIC

Mike Cohn's **test pyramid** (2009) says: many fast unit tests at the base, fewer service-level tests in the middle, very few slow UI tests at the top [21]. Kent C. Dodds's **testing trophy** reshuffles it for modern frontend work: static checks (types, lint) at the base, then unit tests, then the *widest* layer is integration tests, then a few end-to-end tests. His argument is that tests resembling how the software is actually used give more confidence [22]. Neither shape comes from experiments; both are rules of thumb about cost and confidence.

Fomodoro is roughly trophy-shaped: TypeScript and lint as the base; unit tests for pure logic like "what block comes after the third focus?"; component-level tests for screens; a few Playwright tests for the critical paths (start a timer, sign in).

Property-based testing HEURISTIC

Instead of writing individual examples ("25 minutes in → 1500 seconds out"), you state a **property** that should hold for *all* inputs, and a tool generates hundreds of random inputs looking for a counterexample. The idea comes from QuickCheck, a Haskell tool by Koen Claessen and John Hughes [23]; JavaScript has equivalents such as fast-check.

Property: "for any valid settings, the remaining time never goes below zero and never exceeds the block's duration." The tool might find that a 0-minute break, which no one thought to test, breaks it.

Why Heuristic: it's a well-established technique, but I did not verify controlled studies comparing its bug-finding against example-based tests, so it is labeled conservatively.

Why coverage percentage misleads STRONG

Code coverage tells you which lines ran during tests, not whether the tests would notice if those lines were wrong. Laura Inozemtseva and Reid Holmes generated 31,000 test suites for five large Java systems and found that, once you control for the number of tests, coverage is only weakly to moderately correlated with how good a suite is at catching faults [24]. Their advice: coverage is useful for finding *untested* code, but a poor target. The paper won ICSE's Most Influential Paper award ten years later.

A test that renders the timer and asserts nothing gives 100% coverage of the render function and catches zero bugs. That is exactly the kind of test an AI agent can produce when told "get coverage to 90%."

Mutation testing STRONG

Mutation testing measures test quality directly. A tool makes many small deliberate bugs ("mutants") in your code (flip `>` to `>=`, replace a value with 0, delete a line) and reruns your tests against each. If a test fails, the mutant is **killed**; if all tests still pass, it **survived**, which means your tests would not have noticed that bug. The **mutation score** is the percentage killed.

Is a fake bug a fair stand-in for a real one? René Just and colleagues checked against 357 real faults from five Java programs and found that mutant detection is significantly correlated with real-fault detection, even after accounting for coverage [25]. A large survey by Papadakis and colleagues reviews a decade of work on the technique and its pitfalls [26].

Stryker runs on our PRs, and the gate is "mutation score must not go down." A new feature with weak tests lowers the score and blocks the merge until the tests actually check behavior.

Why AI agents weaken tests, and how gates stop it MODERATE

When an agent is told "make the tests pass," the shortest path is sometimes to change the tests rather than the code: loosen an assertion, skip a case, special-case the test input. ImpossibleBench (2025) measured this by giving models coding tasks whose tests deliberately contradicted the written spec, so

any "pass" required cheating [27]. Frontier models often cheated, more capable models tended to cheat *more*, and stricter prompts and restricted test access reduced but did not eliminate it.

Hence our test-integrity check: CI blocks any PR that deletes, skips, or weakens an existing test unless I, the human, apply a label to approve it. Combined with the mutation gate, a weakened test can't slip through quietly. It also means an instruction in AGENTS.md ("don't weaken tests") is backed by something that can't be argued with.

Why Moderate: one benchmark study with artificial tasks. The direction matches widely reported practitioner experience, but the real-world rate is unknown.

6. Continuous integration and quality gates

Continuous integration STRONG

Continuous integration (CI) means every change is merged into mainline frequently and automatically built and tested, so problems surface within minutes of being introduced. Fowler's article lists the core practices: everything in version control, a one-command automated build, a self-testing build, frequent integration, a build on every push, fixing a broken build immediately (often by reverting), and keeping the build fast, around ten minutes [12]. DORA counts CI and test automation among the capabilities that predict delivery performance [1], and the 2025 report singles out strong automated testing as what keeps AI's extra output from becoming instability [5].

Our blocking gates: typecheck, lint, unit tests, build, Playwright e2e, Stryker no-regression, test-integrity, gitleaks secret scanning. Advisory: AI review comments. If the pipeline creeps past about ten minutes, that's a signal to speed it up, not to skip gates.

Preview deployments HEURISTIC

A preview deployment is a temporary, real copy of the site built from a PR branch, with its own URL. It lets you click through a change in a real browser, on a real phone, before merging. It's the "production-like test environment" practice from CI made nearly free by modern hosts [12].

Cloudflare Pages builds a preview for every PR. Before merging the iPhone install sheet change, I open the preview URL in Safari on my phone.

Branch protection HEURISTIC

Branch protection is a repository setting that makes the rules mechanical: no direct pushes to main, required status checks must pass, and (for us) only the human can merge. It turns "we always run CI" from a habit into a fact.

Even if Claude, or I late at night, tries to push straight to main, GitHub refuses.

7. Working with AI agents

Almost everything in this section is new, fast-moving, and mostly vendor or practitioner guidance. Treat it as current best guesses, not settled science.

Context rot MODERATE

A language model's **context** is everything it can "see" at once: instructions, files, conversation so far. Chroma's 2025 technical report tested 18 models and found that performance drops as input gets longer, even on simple tasks such as finding one fact in a long document [28]. Distracting, similar-looking content made it worse. In one test, models did far better with a focused prompt of about 300 tokens than with the full 113,000-token history containing the same answer. They called the effect **context rot**.

A long Claude session that started with a planning discussion, explored three dead ends, and is now implementing will perform worse than a fresh session given only the plan file. That's why we do one issue per session.

Why Moderate: one technical report from a company that sells retrieval tools (a possible conflict of interest), though the code is open and the finding matches what Anthropic and others describe.

Context engineering HEURISTIC

Anthropic's applied AI team describes **context engineering** as curating the smallest set of high-signal information an agent needs, treating context as a limited "attention budget" [29]. Their techniques include loading information just in time (keep file paths, read files when needed), compaction (summarize and restart when context fills), structured notes kept outside the context window, and sub-agents that do focused work and return a short summary.

Our version: a short always-loaded AGENTS.md; rules scoped to file paths so testing rules only load when touching tests; skills loaded on demand; plan files as handoff notes; subagents for exploring the codebase and for fresh-context review.

Advisory instructions vs enforced guardrails HEURISTIC

Anthropic's Claude Code guidance draws a clear line [30]. Instructions in a prompt or an instructions file are **advisory**: the model reads them and usually follows them, but there's no guarantee. Hooks, permission deny rules, and CI are **enforced**: they run every time regardless of what the model decides. The advice: if a rule must never have exceptions, make it a hook or deny rule; if you keep adding "IMPORTANT" to an instruction that still gets missed, it belongs in a hook. Keep the enforced set small, because every enforced rule occasionally fires when it shouldn't and needs maintenance.

> **Our layers, from soft to hard.** AGENTS.md (advisory: style, conventions, how we work) → `.claude/settings.json` permissions and hooks (enforced on the agent: e.g. block edits to `.env`, run formatting after edits) → git pre-commit hooks (enforced on any commit: lint, secrets) → branch protection and CI (enforced on the repo: nothing reaches main without passing gates and a human merge). Each layer catches what the one above lets through.

Human in the loop at decision points HEURISTIC

Rather than approving every keystroke, put the human where the decisions are: approving the spec, approving the plan, approving test changes, and merging. Spec Kit and Kiro both build in exactly these checkpoints [16][17], and the METR result [7] is a reminder that the human's impression of progress needs checking against something concrete.

I approve the spec and plan, approve any test-integrity label, and am the only one who can merge. Between those points Claude works freely inside the guardrails.

The limits of AI code review HEURISTIC

An AI reviewer is useful as a second pair of eyes, especially when it runs in a **fresh context** that never saw the reasoning that produced the code and so isn't swayed by it [30]. But it shares blind spots with the AI that wrote the code, can be confidently wrong, and doesn't know what you *meant*. Bacchelli and Bird's finding applies here too: the core of review is understanding, and only the person accountable for the code can supply that [15].

That's why AI review in our pipeline is advisory, not blocking. Its comments are prompts for my attention, not verdicts.

8. Logs, retrospectives, and handoff

Blameless retrospectives HEURISTIC

A retrospective is a regular pause to ask what went well, what didn't, and what to change. Norm Kerth's "Prime Directive" sets the tone: assume everyone did the best job they could with what they knew at the time [31]. Esther Derby and Diana Larsen's widely used structure has five stages: set the stage, gather data, generate insights, decide what to do, close [32]. Blameless matters even when solo: the useful question is "what in the process let this happen?" not "why was I careless?"

When a merge breaks something, the log entry asks which gate should have caught it. If the answer is "none," the fix may be a new test or a new gate, not just the bug fix.

Dev log and status doc as handoff HEURISTIC

Every merged PR gets a short log entry: what changed, what surprised us, what to watch. Logs serve two readers who have no memory of the work: future you, and every new AI session. Anthropic's context guidance recommends exactly this kind of structured note kept outside the context window [29].

A new Claude session reads PDLC.md and the latest log entries and knows where things stand, without re-reading the whole repo or the previous chat.

Process consolidation HEURISTIC

Process documents rot in a predictable way: every incident adds a rule, nothing is ever removed, and eventually nobody reads it, human or AI. The fix is a hard size cap and regular consolidation.

PDLC.md is capped at 80 lines. Every five merged PRs, the monthly log is reviewed: lessons that held up get folded into PDLC.md, rules that never fired get cut, and the cap forces a choice. This is a retrospective, done on a schedule, with the process doc as its output.

Summary: where the evidence is strongest

Concept	Evidence	In fomodoro

DORA metrics, small batches, CI	STRONG	Auto-deploy on merge; one issue per PR; blocking gates.
Coverage misleads; mutation testing	STRONG	Stryker no-regression gate instead of a coverage target.
AI amplifies; METR trial; TDD; review size and findings; trunk-based; context rot; AI test-gaming	MODERATE	Gates before speed; measure via the log; short cycles; small PRs; fresh sessions; test-integrity check.
Walking skeleton, tracer bullets, specs, ADRs, test shapes, property tests, previews, branch protection, context engineering, guardrail layers, human checkpoints, AI review limits, retros, logs	HEURISTIC	M0; spec → plan → build; layered guardrails; PDLC.md cap.

Sources

1. Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps.* IT Revolution Press.
2. DORA. (n.d.; updated January 2026). DORA's software delivery performance metrics. https://dora.dev/guides/dora-metrics/
3. DORA. (n.d.). Working in small batches. DORA Capabilities. https://dora.dev/capabilities/working-in-small-batches/
4. DORA / Google Cloud. (2025). *State of AI-assisted Software Development* (2025 DORA Report). https://dora.dev/dora-report-2025/
5. Harvey, N., & DeBellis, D. (2025, September 23). Announcing the 2025 DORA Report. Google Cloud Blog. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report
6. Becker, J., Rush, N., Barnes, B., & Rein, D. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv:2507.09089. https://arxiv.org/abs/2507.09089
7. METR. (2025, July 10; page updated since). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [blog post]. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/
8. Cockburn, A. (2004). *Crystal Clear: A Human-Powered Methodology for Small Teams.* Addison-Wesley. (Strategy: Walking Skeleton.)
9. Freeman, S., & Pryce, N. (2009). *Growing Object-Oriented Software, Guided by Tests.* Addison-Wesley. Ch. 4, "First, Test a Walking Skeleton"; Ch. 10, "The Walking Skeleton."
10. Thomas, D., & Hunt, A. (2019). *The Pragmatic Programmer: Your Journey to Mastery* (20th anniversary ed.). Addison-Wesley. Topic 12, "Tracer Bullets." (1st ed. Hunt & Thomas, 1999.)
11. DORA. (n.d.). Trunk-based development. DORA Capabilities. https://dora.dev/capabilities/trunk-based-development/
12. Fowler, M. (2024, revised January 18). Continuous Integration. martinfowler.com. https://martinfowler.com/articles/continuousIntegration.html
13. Cohen, J. (2006). *Best Kept Secrets of Peer Code Review.* Smart Bear Inc. (Ch. 5: Cisco MeetingPlace case study.)

14. SmartBear. (n.d.). Best Practices for Code Review. https://smartbear.com/learn/code-review/best-practices-for-peer-code-review/
15. Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. *Proc. 35th International Conference on Software Engineering (ICSE)*, 712–721. doi:10.1109/ICSE.2013.6606617
16. Delimarsky, D. (2025, September 2). Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/
17. Yanchuleff, R. (2025, July 15). From chat to specs: a deep dive into AI-assisted development with Kiro. Kiro Blog. https://kiro.dev/blog/from-chat-to-specs-deep-dive/
18. Nygard, M. (2011, November 15). Documenting Architecture Decisions. Cognitect Blog. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
19. Rafique, Y., & Mišić, V. B. (2013). The Effects of Test-Driven Development on External Quality and Productivity: A Meta-Analysis. *IEEE Transactions on Software Engineering*, 39(6), 835–856. doi:10.1109/TSE.2012.28
20. Fucci, D., Erdogmus, H., Turhan, B., Oivo, M., & Juristo, N. (2017). A Dissection of the Test-Driven Development Process: Does It Really Matter to Test-First or to Test-Last? *IEEE Transactions on Software Engineering*, 43(7), 597–614. doi:10.1109/TSE.2016.2616877
21. Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum.* Addison-Wesley. (Test automation pyramid.)
22. Dodds, K. C. (n.d.). The Testing Trophy and Testing Classifications. https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications
23. Claessen, K., & Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *Proc. ICFP '00*, ACM SIGPLAN Notices 35(9), 268–279. doi:10.1145/357766.351266
24. Inozemtseva, L., & Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. *Proc. 36th ICSE*, 435–445. doi:10.1145/2568225.2568271
25. Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., & Fraser, G. (2014). Are mutants a valid substitute for real faults in software testing? *Proc. 22nd ACM SIGSOFT FSE*, 654–665.
26. Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, 112, 275–378. doi:10.1016/bs.adcom.2018.03.015
27. Zhong, Z., Raghunathan, A., & Carlini, N. (2025). ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. arXiv:2510.20270 (ICLR 2026). https://arxiv.org/abs/2510.20270
28. Hong, K., Troynikov, A., & Huber, J. (2025, July 14). Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma Technical Report. https://www.trychroma.com/research/context-rot
29. Rajasekaran, P., Dixon, E., Ryan, C., & Hadfield, J. (2025, September 29). Effective context engineering for AI agents. Anthropic Engineering. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
30. Anthropic. (n.d.; accessed November 2026). Best practices for Claude Code. Claude Code Docs. https://code.claude.com/docs/en/best-practices (successor to the April 2025 post "Claude Code: Best practices for agentic coding.")
31. Kerth, N. L. (2001). *Project Retrospectives: A Handbook for Team Reviews.* Dorset House.
32. Derby, E., & Larsen, D. (2006). *Agile Retrospectives: Making Good Teams Great.* Pragmatic Bookshelf. (2nd ed. with D. Horowitz, 2024.)

All sources were checked against publisher, author, or official pages in November 2026. Book chapter content (Cockburn, Freeman & Pryce, Hunt & Thomas, Cohn, Derby & Larsen, Kerth) was confirmed through tables of contents and secondary descriptions rather than the full text.